

ESE 507: Advanced Digital System Design and Generation

Fall 2025

Member: Jing Jin, Xiaoying Li

Course Project

Hardware Accelerator for 2D Convolution

Part 1: MAC Unit

1.1 Unpipelined MAC: Synthesis Results and Analysis

Table 1.1: Synthesis Results for Unpipelined MAC (INW=16, OUTW=64)

Period (ns)	Slack Status (MET)	Frequency (MHz)	Area (μm^2)	Power (μW)	Critical Path
1.5	0	666.67	2105.66	1422.6	Input → Multiplier → Adder → Accumulator Register (accReg)
1.7	0.01	588.24	2057.51	1227.3	
2	0.01	500	2041.28	1050.1	
2.2	0.13	454.55	2048.73	960.07	
2.5	0	400	2059.90	880.06	
3	0.02	333.33	2007.50	702.98	
3.5	0	285.71	1974.78	590.86	
4	0	250	1959.89	511.99	
4.5	0	222.22	1817.84	447.92	
5	0	200	1759.32	400.03	
6	0.5	166.67	1726.07	324.19	
7	1.5	142.86	1726.61	283.35	
8	2.5	125	1726.61	252.73	
9	3.5	111.11	1726.61	228.92	
10	4.5	100	1726.61	209.87	
20	14.5	50	1726.61	124.15	

The critical path starts at input, passes through the multiplier that computes the product $\text{input0} \times \text{input1}$. Then the adder accumulates the multiplier output with the value stored in the accumulator register, and ends at the data input of the accumulator register (accReg).

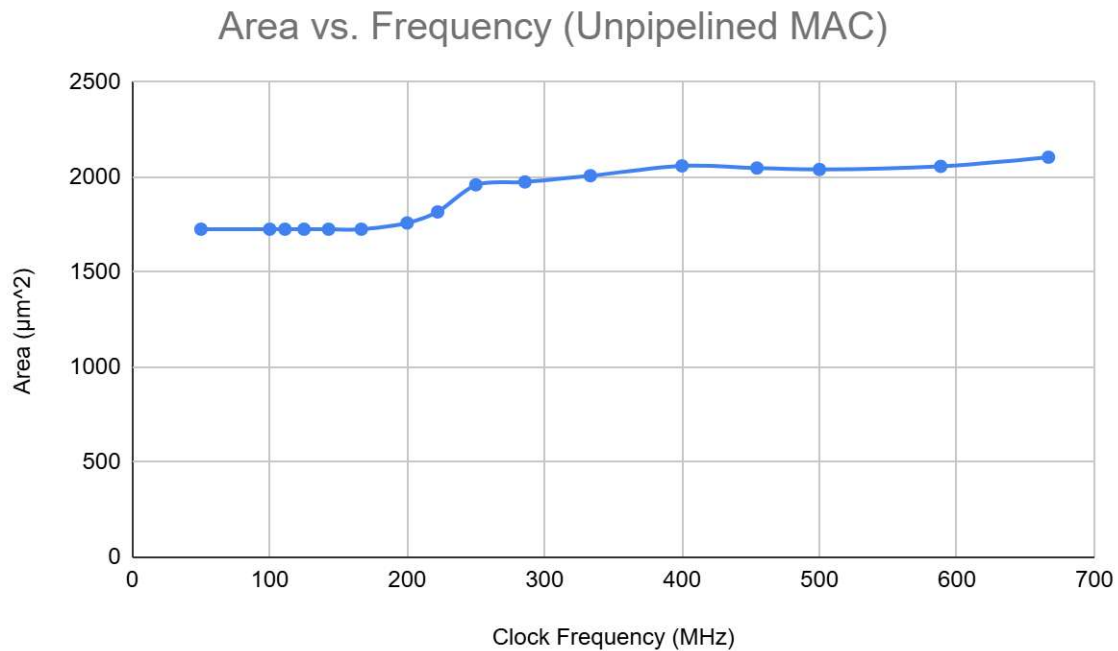


Figure 1.1. Relationship between area and frequency for unpipelined MAC

Area stabilizes at approximately $1726.61 \mu\text{m}^2$ for frequencies below 200 MHz. As frequency increases from 200 MHz to 666.67 MHz, the area rises to $2105.66 \mu\text{m}^2$. Area increases most rapidly between 200 MHz and 250 MHz. Beyond this point, area tends toward stability which implies the design is approaching its fundamental speed limit. At higher frequencies, the synthesis tool wants to meet the timing constraints but leads to area increases. This is because the tool uses faster implementations and higher drive strength to reduce delay.

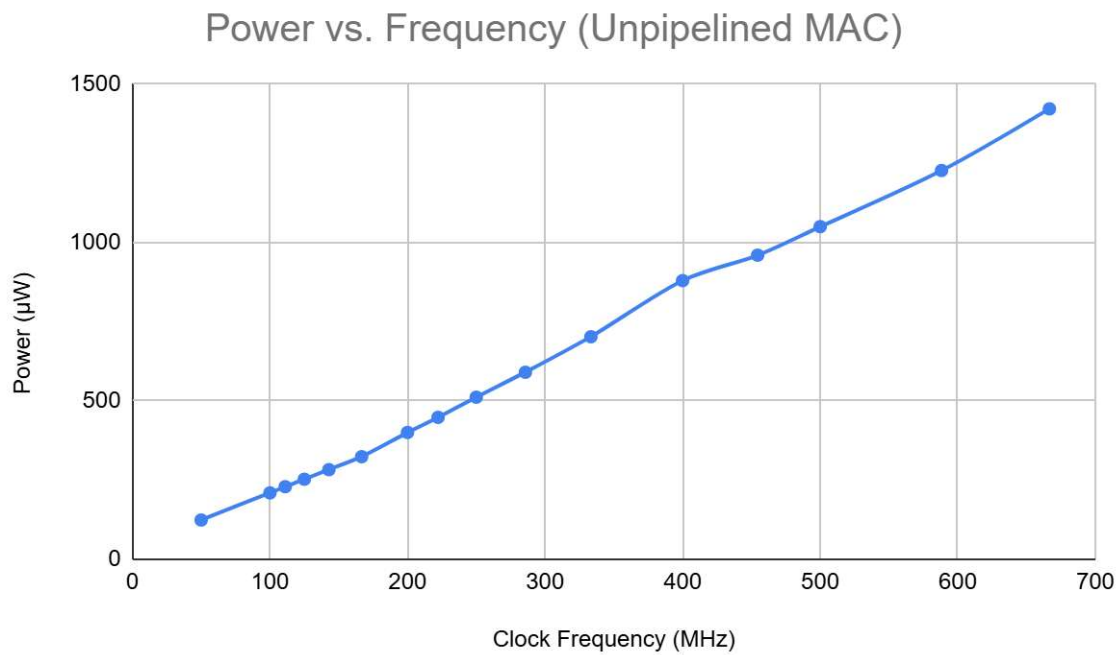


Figure 1.2. Relationship between power and frequency for pipelined MAC

Power scales with clock frequency. It increases from 124.15 μW at 50 MHz to 1422.6 μW at 666.67 MHz. Below 250 MHz, power increases slightly, but past 250 MHz, increases faster due to larger area and higher drive strength pushing power higher.

1.2 Pipelined MAC: Synthesis Results and Analysis

Table 1.2: Synthesis Results for Pipelined MAC (INW=16, OUTW=64)

Period (ns)	Slack Status (MET)	Frequency (MHz)	Area (μm^2)	Power (μW)	Critical Path
1.3	0	769.23	2343.19	1876.8	Input $\rightarrow$ Multiplier $\rightarrow$ pipeReg
1.5	0	666.67	2314.47	1589.4	
1.7	0	588.24	2298.24	1399.9	
2	0.01	500	2247.97	1141.5	
2.5	0	400	2292.39	940.51	
3	0	333.33	2165.24	728.39	
3.5	0	285.71	2124.54	618.49	accReg (out) $\rightarrow$ Adder $\rightarrow$ accReg (in)
4	0.01	250	2004.84	535.02	
4.5	0	222.22	1964.94	474.73	
5	0.01	200	1947.65	428.41	
6	0.51	166.67	1904.83	356.58	
7	1.51	142.86	1906.69	311.17	
8	2.51	125	1906.69	277.38	
9	3.51	111.11	1906.69	251.09	
10	4.51	100	1906.69	230.07	
20	14.51	50	1906.69	135.45	

At higher frequencies (333.33 MHz to 769.23 MHz), the critical path runs from the input, passes through a multiplier (input0*input1), and ends at the input of the pipeline register (pipeReg). In this region, only the multiplier stage determines the max frequency.

At lower frequencies (50 MHz to 285.71 MHz), the critical path begins at output of the accumulator register (accReg), passes through the adder accumulates the product pipeline register output and the accumulator register output, and ends at the input to the accumulator register. In this region, the accumulator feedback loop dominates timing.

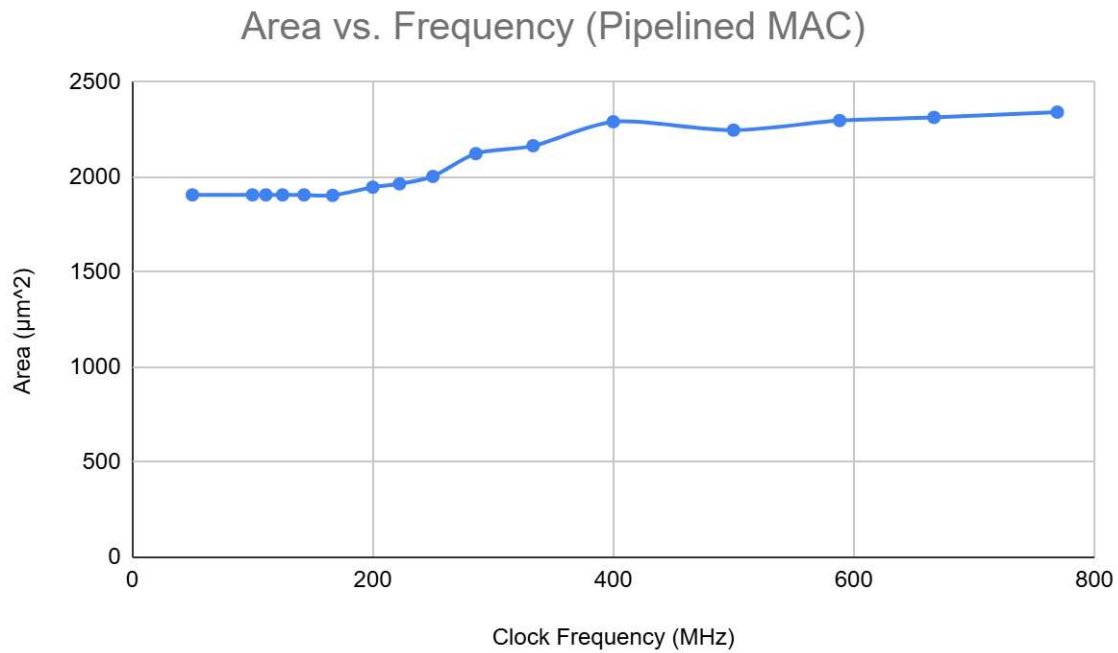


Figure 1.3. Relationship between area and frequency for pipelined MAC

The pipelined MAC has a similar area trend as the unpipelined design but with a higher baseline. This is because of the additional pipeline register and its control logic, the tool upsizes cells and uses faster implementations to meet timing constraints. Below 200 MHz, the area stabilizes around $1906.69 \mu\text{m}^2$. Area increases rapidly between 250 MHz and 400 MHz, and reaches $2343.19 \mu\text{m}^2$ at its maximum frequency 769.23 MHz.

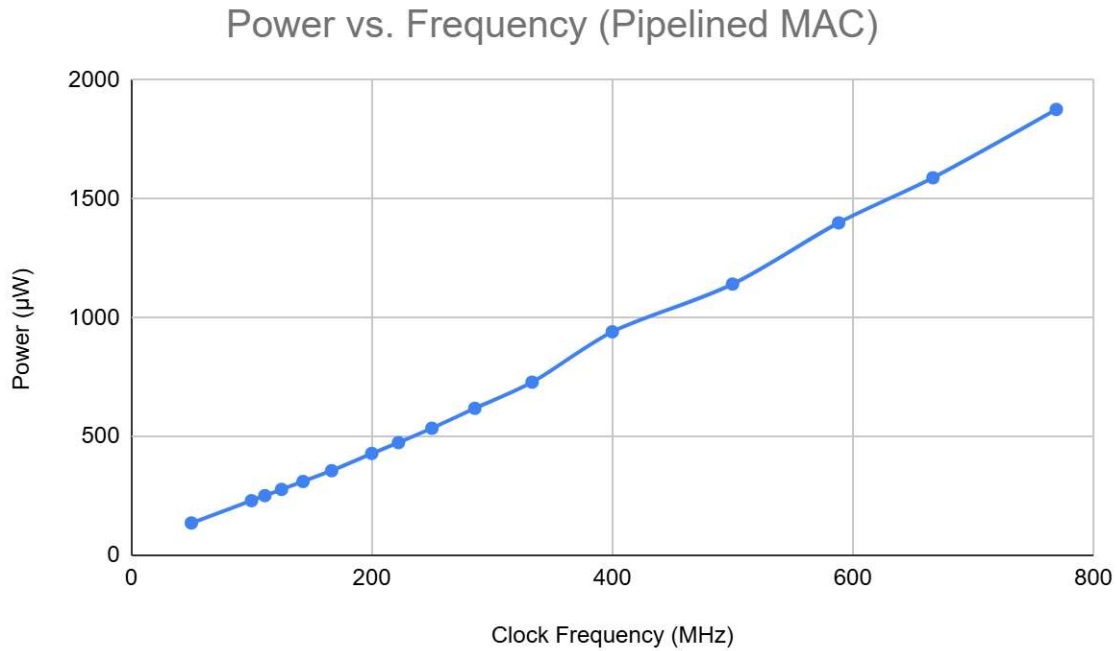


Figure 1.4. Relationship between power and frequency for pipelined MAC

Power increases from 135.45 μW at 50 MHz to 1876.8 μW at 769.23 MHz, and consumes more power than the unpipelined design at equivalent frequencies due to extra registers and control logic.

Pipelining raises the maximum frequency from 666.67 MHz to 769.23 MHz. It splits the original critical path into a multiplier stage and an accumulator stage. At high frequencies, the multiplier stage limits performance, and at low frequencies, the accumulator feedback path dominates. The trade-off is the cost of larger area, higher power, and an extra cycle of latency.

1.3 Energy Consumption at Max Frequency

For $\text{INW}=16$ and $\text{OUTW}=64$, the pipelined MAC reaches its maximum clock frequency at 769.23 MHz (1.3 ns), with power of 1876.8 μW . Since pipelined MAC takes 2 clock cycles for an output, 50 input sets would require 51 cycles to produce output.

Thus,

$$\text{Energy} = \text{Power} * \text{Time} = (1876.8 \mu\text{W})(51 * 1.3 \text{ ns}) = 124.43 \text{ pJ}$$

1.4 Energy Scaling with Frequency

Table 1.3: Energy Scaling with Frequency

Frequency (MHz)	Period (ns)	Power (μ W)	Total Time for 50 Input sets (ns)	Energy (pJ)
50	20	135.45	1020	138.15
100	10	230.07	510	117.33
200	5	428.41	255	109.24
333.33	3	728.39	153	111.44
500	2	1141.5	102	116.43
769.23	1.3	1876.8	66.3	124.43

As clock frequency increases, power consumption also increases. However, energy first decreases with decreasing frequency then increases. For 50 input sets, energy is minimized at approximately 200 MHz. Static power mostly depends on area, it dominates at lower frequencies and energy increases per task. While dynamic power is proportional to frequency, it dominates at higher frequencies and energy increases again.

1.5 Performance Comparison: Unpipelined vs. Pipelined

Table 1.4: Performance Metrics at Maximum Clock Frequency

MAC	Period (ns)	Frequency (MHz)	Power (μ W)	Area (μm^2)	Latency (ns)	Throughput (billion inputs per sec.)
Unpipelined	1.5	666.67	1422.6	2105.66	1.5	1.33
Pipelined	1.3	769.23	1876.8	2343.19	2.6	1.54

Unpipelined MAC calculations:

- Latency: 1 cycle * 1.5 ns = 1.5 ns
- Throughput: 2 inputs/cycle * 666.67 MHz = 1.33 billion inputs/sec

Pipelined MAC calculations:

- Latency: 2 cycles * 1.3 ns = 2.6 ns
- Throughput: 2 inputs/cycle * 769.23 MHz = 1.54 billion inputs/sec

Pipelined MAC has higher frequency and offers higher throughput, making it better for high volume streaming applications. Unprepared MAC has smaller area and power consumption, and lower latency, more suitable for power or area constrained designs.

1.6 Deep Pipelining

Breaking the multiplier into multiple stages would increase clock frequency by shortening the critical path. This approach adds additional registers, increases area, and produces extra latency. Pipelining the adder is not beneficial. Because the adder is inside the accumulator feedback loop, inserting a pipeline stage would break this loop and require extra cycles before accumulated results, also increasing control complexity and hurting functionality. Moreover, at higher frequencies, the critical path is the multiplier stage, not the adder. Thus, pipelining the adder would not improve maximum frequency.

1.7 Sensitivity to INW and OUTW

Table 1.5: Synthesis Results with Parameter Variation

INW	OUTW	Period (ns)	Frequency (MHz)	Critical path
12	24	1.06	943.40	Input → Multiplier → pipeReg (in)
12	48	1.05	952.38	
12	64	1.07	934.58	
8	48	0.86	1162.79	
16	48	1.17	854.70	
24	48	1.49	671.14	

The critical path is identical for all tested parameters. It starts at input, passes through the multiplier (input0*input1), and ends at the input to the pipeline register (pipeReg).

Frequency varies minimally as OUTW changes. OUTW affects adder in the feedback loop, but adder is not on the critical path. When INW is fixed, the multiplier output bit width ($2 \times \text{INW} = 24\text{-bit}$) remains the same, so output width grows from 24-bit to 64-bit isn't critical and doesn't affect maximum frequency. In contrast, frequency decreases as INW increases, because

INW directly affects the multiplier. Multiplier is on the critical path, larger INW leads to longer critical path and lower maximum frequency. Overall, changing OUTW yields limited improvement, while reducing INW improves performance.

1.8 MAC Overflow

For signed 5-bit input (INW), max values are 15 and -16. The largest magnitude product is $(-16)*(-16) = 256$ (10-bit). For a 16-bit accumulator (OUTW), maximum positive and negative values are 32767 and -32768. Starts with initialization value 15 and repeatedly adding 256. The maximum number of MACs without overflow is $(32767-15) / 256 = 127$.

Verification:

- After 127 MACs: $15 + 127*256 = 32527$
- After 128 MACs: $32527 + 256 = 32783$ (overflow occurs)

1.9 Contribution

Xiaoying drafted the code and completed the report section. Jing identified and fixed code errors.

Part 2: Output FIFO

2.1 AXI-Stream FIFO Interface Design

The basic FIFO design uses enable signals and a capacity counter. The FIFO module adapts this design to support the AXI-Stream interface and implements the handshake protocol with `TVALID` and `TREADY` signals.

On the input side, a write happens when both `IN_AXIS_TVALID` (from transmitter) and `IN_AXIS_TREADY` (from FIFO) are asserted. `IN_AXIS_TREADY` is driven by FIFO's full status. When the FIFO is not full, `IN_AXIS_TREADY` is asserted, which allows the FIFO to accept new data. The write enable signal (`wr_en`) is generated when both `TVALID` and `TREADY` are high. In this case, the data on `IN_AXIS_TDATA` is written to the memory address pointed to by the head pointer, and the head pointer increments (with wrap-around at `DEPTH`). If the FIFO becomes full, `IN_AXIS_TREADY` is deasserted, this prevents the transmitter from writing until space becomes available.

Similarly, on the output side, a read is controlled by `OUT_AXIS_TVALID` (from FIFO) and `OUT_AXIS_TREADY` (from receiver). The FIFO asserts `OUT_AXIS_TVALID` when it is not empty, indicating that valid data is available on `OUT_AXIS_TDATA`. When both `TVALID` and `TREADY` on the output side are high, the read enable (`rd_en`) is generated, and the tail pointer advances to read the next value. If the receiver is not ready (`OUT_AXIS_TREADY=0`), no read occurs and data remains available in the FIFO.

A capacity counter tracks the current number entries in the FIFO. When the counter reaches 0, the FIFO is empty, deasserts `OUT_AXIS_TVALID`. When the counter reaches `DEPTH`, the FIFO is full, deasserts `IN_AXIS_TREADY`. Counter increments when a write occurs, and decrements when a read occurs. If read and write occur at the same rising edge, the counter remains unchanged.

2.2 Baseline Synthesis Results: `OUTW = 24`, `DEPTH = 19`

- Minimum clock period = 0.93 ns
- Maximum clock frequency = 1.0753 GHz
- Area = 3967.124 μm^2
- Power = 2506.4 μW

- Critical path: capacity_reg (out) → comparator → handshake logic (rd_en) → tail logic (rd_addr) → memory module (data_out)

The critical path starts at the output of the capacitor counter, passes through the empty comparator and handshake logic to generate the read enable (rd_en). Then proceeds through the tail pointer increment logic (with wrap-around) to produce a read address (rd_addr). Finally the computed read address signal goes to the memory module and selects the valid memory location to output (mem[rd_addr]). This is a memory read path.

2.3 Parameter Sweep and Performance Analysis

Table 2.1: Parameter Sweep Results

Parameters	Period (ns)	Frequency (GHz)	Area (μm^2)	Power (μW)	Critical Path
OUTW = 12, DEPTH = 19	0.91	1.0989	2317.924	1452.0	capacity_reg (out) → comparator → handshake logic (rd_en) → tail logic (rd_addr) → memory module (data_out)
OUTW = 24, DEPTH = 19	0.93	1.0753	3967.124	2506.4	
OUTW = 24, DEPTH = 38	0.96	1.0417	7494.816	5242.7	

The critical path is the same for these three configurations. It starts at capacity counter output, passes through the empty comparator, handshake logic (rd_en) and tail pointer logic (rd_addr), and finally into the memory module to select the output.

Maximum Frequency:

- OUTW: $(1.0989 - 1.0753) / 1.0989 = 2.15\%$ decreases
- DEPTH: $(1.0753 - 1.0417) / 1.0753 = 3.12\%$ decreases

When OUTW doubles from 12 to 24, it slightly decreases by 2.15 %. This is because OUTW only affects data width and not on the critical path. The critical path goes through control logic: capacity comparison, handshake generation, tail pointer update, and none of these depend on the output data width. On the other hand, when DEPTH doubles from 19 to 38, it slightly decreases with some larger impact. Larger DEPTH increases the number of address bits, from 5 bits to 6 bits. Overall, both parameters produce minimal frequency impact.

Area:

- OUTW: $(3967.124-2317.924)/2317.924 = 71.15\%$ increases
- DEPTH: $(7494.816-3967.124)/3967.124 = 88.92\%$ increases

When OUTW doubles, area increases by 71.15 % due to memory sizing. The total memory bits double from 228-bit (19*12) to 456-bit (24*19), approximately doubles the required storage.

When DEPTH doubles, the area increases by 88.92 %. This is again because of memory size, increases pointer width, adds overhead to control logic, and larger the memory.

Power:

- OUTW: $(2506.4-1452)/1452 = 72.62\%$ increases
- DEPTH: $(5242.7-2506.4)/2506.4 = 109.17\%$ increases

Since power scales with area, when OUTW doubles, it is proportional to the number of registers (introduce more capacitance). When DEPTH doubles, power is more than doubling because DEPTH affects both memory size and control logic. Double memory size meaning more memory related power. Additionally, wider pointers and larger addresses require more internal switching and more area consumption. Combined effects cause power scale more aggressively than area when DEPTH increases.

2.4 Contribution

Xiaoying drafted the code and completed the report section. Jing reviewed and verified the code.

Part 3: Input Memory

3.1 The Input Memory Module Documentation

The `input_mems` module manages the loading and storing of all input required for computation: weight matrix `W`, the bias value `B`, and input matrix `X`. It consists of memories, counters, registers, address multiplexing, and a FSM that controls all data movement.

Module Architecture

Memory Submodules:

- `W Memory (wmem)`: stores weight matrix elements with size $\text{MAXK} \times \text{MAXK}$.
- `X Memory (xmem)`: stores input matrix elements with size $R \times C$.

Both memories support writes during the load phase and reads during the compute phase through address multiplexing. During the load phase (`inputs_loaded=0`), memories use internal write counters (`w_cnt` for `W`, `x_cnt` for `X`). During the compute phase (`inputs_loaded=1`), memories use external read addresses (`W_read_addr`, `X_read_addr`).

Control Logic:

- `AXI-Stream Handshake (fire)`: a valid data transfer occurs when both `AXIS_TVALID` and `AXIS_TREADY` are high. During loading phases, `AXIS_TREADY` remains asserted except in the `WAIT_COMPUTE` state (new inputs are blocked).
- `AXIS_TUSER Decode`: this field is only valid on the first beat of each transaction.
 - `new_W`: extracts from `AXIS_TUSER[0]`, determines loading path. When `new_W=1`, load `W` and `B`. When `new_W=0`, reuse previous `W` and `B`.
 - `TUSER_K`: extracts from `AXIS_TUSER[K_BITS:1]`, provides the kernel size of `K` when `new_W=1`.
- `Write Control`:
 - `W_wr_en`: asserts when handshake occurs in `INPUT_W` or in `IDLE` with `new_W=1`.
 - `X_wr_en`: asserts when handshake occurs in `INPUT_X` or in `IDLE` with `new_W=0`.

- **Registers and Counters:**
 - The K Register stores the current kernel size which is captured from `TUSER_K`. It updates only when `new_W=1` during the first beat of each transaction. For `new_W=0`, K retains its previous value.
 - The B Register stores the current bias value, which is captured from `AXIS_TDATA` during `INPUT_B` state when handshake occurs. For `new_W=0`, B retains its previous value.
 - The `w_total` Register stores the total number of weights for the current transaction ($K*K$ elements). It updates when `new_W=1` calculated as $TUSER_K * TUSER_K$ on the first beat.
 - The `w_cnt` Counter tracks number of weight elements written to W memory, ranging from 0 to `w_total` ($K*K$). Increments on each weight write and resets outside `INPUT_W`. When the first element is written in IDLE, `w_cnt` starts at 1.
 - The `x_cnt` Counter tracks number of input elements written to X memory, ranging from 0 to $R*C$. Increments on each input write and resets outside `INPUT_X`. When the first element is written in IDLE, `x_cnt` starts at 1.

FSM States:

- **IDLE:** initial state after reset or `compute_finished`. The state monitors `AXIS_TVALID` to begin a new transaction. On the first handshake, examine `new_W` to determine the loading path:
 - If `new_W=1`, writes `W[0]` to W memory, captures K and `w_total` from `TUSER_K`, sets `w_cnt` to 1, and transitions to `INPUT_W`.
 - If `new_W=0`, writes `X[0]` to X memory, reuses previous K and B, sets `x_cnt` to 1, and transitions to `INPUT_X`.
- **INPUT_W:** sequentially loads weight values into W memory in row-major order. Each handshake increments `w_cnt` and writes `AXIS_TDATA` to address `w_cnt`. Transitions to `INPUT_B` when `w_cnt` reaches `w_total-1` (after all $K*K$ weight elements loaded).
- **INPUT_B:** loads the single bias value B, captures `AXIS_TDATA` into B register, then transitions to `INPUT_X` after one handshake.

- **INPUT_X**: sequentially loads input feature map values into X memory in row-major order. Each handshake increments `x_cnt` and writes `AXIS_TDATA` to address `x_cnt`. Transitions to `WAIT_COMPUTE` when `x_cnt` reaches `R*C-1` (after all `R*C` input elements loaded).
- **WAIT_COMPUTE**: during the compute phase, all input data are loaded. Asserts `inputs_loaded` to signal data is ready for computation, and deasserts `AXIS_TREADY` to block new inputs. Memory addresses switch from write counters to external read addresses. Additionally, monitor `compute_finished` signal, when it asserts:
 - If `AXIS_TVALID` is high, immediately processes the first beat based on `new_W`. Transitions to `INPUT_W` (if `new_W=1`) or `INPUT_X` (if `new_W=0`).
 - If `AXIS_TVALID` is low, return to `IDLE`.

Operation Modes

- **Loading New Weights and Bias (`new_W=1`)**: the system starts in `IDLE`. When the first beat occurs with `new_W=1`, writes `W[0]` to W memory, captures K and `w_total`, and FSM transitions to `INPUT_W` with `w_cnt=1`. The FSM then proceeds through `INPUT_W`, `INPUT_B`, `INPUT_X`, and `WAIT_COMPUTE`.
- **Reusing Weights and Bias (`new_W=0`)**: the system starts in `IDLE`. when the first handshake occurs with `new_W=0`, write `X[0]` to X memory, K and B retain previous values, and FSM transitions to `INPUT_X` with `x_cnt=1`. The FSM proceeds `INPUT_X` then `WAIT_COMPUTE`.
- **Back-to-Back Transitions**: when FSM in `WAIT_COMPUTE`, both `compute_finished` and `AXIS_TVALID` are asserted, the system immediately processes the first beat for the next transaction based on the new value of `new_W` (without returning to `IDLE`).

3.2 Cycle Count and Efficiency

Run simulation with parameters INW=10, R=15, C=13, MAXK=7, INPUT_VALID_PROB=1, and measure cycles counts when new_W equals 0 and 1.

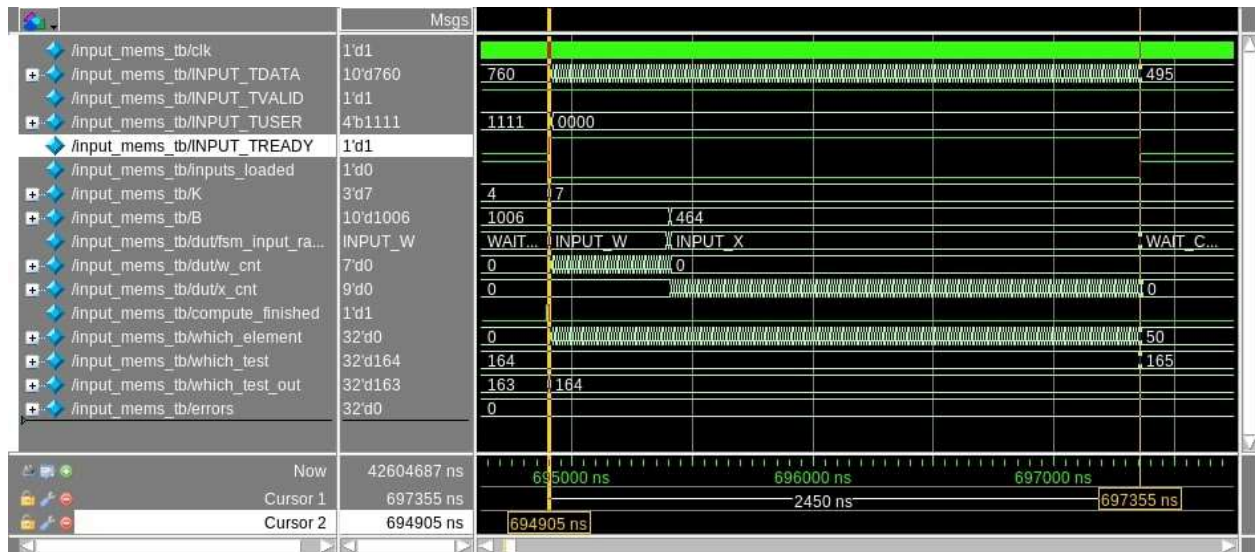


Figure 3.1. Cycle Count with new_W=1

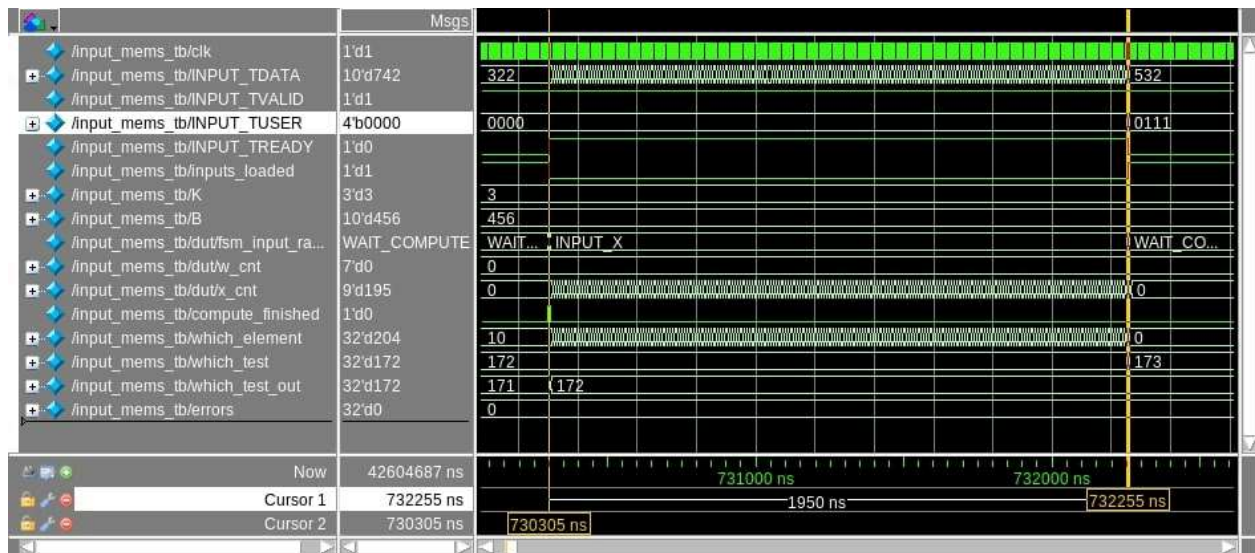


Figure 3.2. Cycle Count with new_W=0

Table 3.1: Cycle Count for Input Loading

new_W	K	Expected Cycles	Measured Cycles	Efficiency
1	7	245	245	1
	2	200	200	1
	6	232	232	1
0	3	195	195	1
	6	195	195	1
	5	195	195	1

Cycle count calculations:

For new_W=1:

$$\text{Efficiency} = (K*K+1+R*C) / \text{cycles} = (K*K+196) / \text{cycles}$$

For new_W=0:

$$\text{Efficiency} = (R*C) / \text{cycles} = 195 / \text{cycles}$$

All measured efficiencies equal 1 which is the perfect efficiency. Meaning the system has no unnecessary stalls with no extra wait cycles.

3.3 Cycle Count Equations

These equations assume INPUT_TVALID_PROB=1, with lower valid probabilities, additional cycles take place.

For new_W=1:

$$\text{cycle count} = K*K + 1 + R*C$$

For new_W=0:

$$\text{cycle count} = R*C$$

3.4 Synthesis Analysis

Table 3.2: Synthesis Results for Input Memory

Parameters	Period (ns)	Frequency (GHz)	Area (μm^2)	Power (μW)	Critical Path
INW=24, R=9, C=8, MAXK=4	0.72	1.3889	16411.668	17698	FSM cs_Reg (WAIT_COMPUTE) → Multiplexer (X_read_addr) → X Memory (xmem)
INW=10, R=15, C=13, MAXK=7	0.81	1.2346	17830.512	20113	

The critical path starts at the FSM (WAIT_COMPUTE state), passes through the address multiplexer (X_addr) that selects external read address (X_read_addr). The selected read address then enters the X memory (xmem), accesses the correct memory location and outputs the stored data (mem[addr]).

Both designs have the same critical path. The parameters affect memory sizes and data widths, but not the fundamental datapath. The critical path remains as the memory read path, it goes through the control logic and address multiplexer for address selection, not affecting data width or memory size. Overall, changing in INW affects data width, while changing R, C, and MAXK affect memory size.

3.5 Contribution

Xiaoying drafted the code and completed the report section. Jing optimized the code and verified correctness.

Part 4: 2D Convolution

4.1 The Conv Documentation

The design is working in a serial way: the system first loads all weights, bias, and the input feature map over AXI, then computes each output one by one by repeatedly reading the required $K \times K$ data and feeding a single mac_pipe, and finally pushes the results into the output FIFO, with these phases happening strictly one after another and never overlapping.

4.1.1 AXI input stage

1. Over the AXI-Stream interface, the module receives data in a fixed order:
 - First: the weights W (size $K \times K$).
 - Then: the bias B .
 - Finally: the input feature map X (size $R \times C$).
2. As each word arrives from AXI, it is written into internal storage:
 - W is written into a weight memory.
 - B is stored in a separate bias register.
 - X is written into an input memory (a RAM that stores the entire $R \times C$ matrix).
3. Only after all data has been received ($W \rightarrow B \rightarrow X$), the input module:
 - Asserts an `inputs_loaded` signal to the next stage.
 - Then it essentially stops and waits: it does not accept or process a new transaction until the current computation is completely finished.
4. The downstream compute module runs using this fixed snapshot of W , B , and X .
5. When the compute module has processed all outputs, it asserts a `compute_finished` signal back to the input/control logic, indicating that this batch is done and the system can start receiving the next set of $W/B/X$.

4.1.2 Control + MAC stage – fully serial MAC per output

The second part is the control logic + MAC datapath that actually performs the convolution or matrix operation in a fully serial MAC loop.

1. For each output element $Y[i,j]$ (for all valid i,j):
 - The control logic computes the addresses of the $K \times K$ window in X and the corresponding weights in W .

- It then reads one pair of operands at a time: one element from X, one element from W. The read command continues for $K \times K$ cycles.
 - These operands are fed into a single MAC pipeline (mac_pipe), which performs: a multiply plus an accumulation into an internal accumulator (acc_reg).
2. The output process is fully serial:
 - For one output $Y[i,j]$, the system needs $K \times K$ multiply-accumulate operations.
 - The control logic loops $K \times K$ times:
 - For each cycle: read one X value and one W value, send them into mac_pipe, update the partial sum.
 - Only after the last (the $K \times K$ -th) pair is sent into the MAC pipeline, this output's accumulation is complete (after the pipeline latency).
 3. Because the MAC is pipelined (mac $\rightarrow$ pipe_reg $\rightarrow$ acc_reg $\rightarrow$ fifo_in), after sending the last MAC input for this output, the control logic must wait for the MAC pipeline to drain (wait a cycle for the result to come out of mac_pipe and reach the FIFO input).
 4. Once the final accumulated result (plus bias) is ready, it is passed to the output FIFO.
 5. Then the controller moves on to the next output element and repeats the same process:
 - Re-read the corresponding $K \times K$ elements from X memory
 - Re-read the $K \times K$ weights from W memory
 - Run through the MAC loop again, and wait for the MAC pipeline to produce the next result.

4.2 Cycle Count

1. Input loading stage

In the original serial design, the input phase does this once per run:

- When new_W==1:
 - read K^2 weight values (state INPUT_W) $\rightarrow K^2$ cycles
 - read 1 bias value (state INPUT_B) $\rightarrow + 1$ cycle
 - Read the $R \cdot C$ input pixels (state INPUT_X) $\rightarrow + R \cdot C$ cycles

So the input time is

$$T_{in}^{new} = K^2 + 1 + R \cdot C$$

- When $\text{new_W}==0$, we skip INPUT_W and INPUT_B, and only read the image X:

$$T_{\text{in}}^{\text{reuse}} = R \cdot C$$

These give you the first term in each formula.

2. Compute stage

- There are N output pixels:

$$N_{\text{out}} = (R - K + 1)(C - K + 1)$$

For each output pixel, the controller:

- reads K^2 pairs of (X, W) and sends them into the MAC pipeline, roughly K^2 cycles per output;
- then waits for the MAC pipeline latency $L_{\text{mac}} = 1$ once per run.

So the total compute time is

$$T_{\text{comp}} \approx N_{\text{out}} \cdot K^2 + L_{\text{mac}} = (R - K + 1)(C - K + 1) \cdot K^2 + L_{\text{mac}}$$

$$L_{\text{mac}} = 1$$

3. Total cycle count

Because the original design is fully serial (first load everything, then compute everything), the total time is just(:

- $\text{new_W}==1$:

$$T_{\text{total}}^{\text{new}} \approx (K^2 + 1 + R \cdot C) + (R - K + 1)(C - K + 1) \cdot K^2 + L_{\text{mac}}$$

$$L_{\text{mac}} = 1$$

- $\text{new_W}==0$:

$$T_{\text{total}}^{\text{reuse}} \approx R \cdot C + (R - K + 1)(C - K + 1) \cdot K^2 + L_{\text{mac}}$$

$$L_{\text{mac}} = 1$$

4.3 2D Convolution Synthesis Results

Table 4.1: Synthesis Results for Convolution Module

Parameters	Period (ns)	Frequency (MHz)	Area (μm^2)	Power (mW)
INW=12, R=9, C=8, MAXK=5	1.3	769.23	12700.17	7.51
INW=18, R=9, C=8, MAXK=5	1.6	625	18894.51	9.23
INW=24, R=16, C=17, MAXK=9	1.8	555.56	74787.76	37.36

The critical path is identical across all configurations, it goes from the weight-memory output register to the MAC pipeline register and passes almost entirely through the synthesized multiplier logic (a long chain of full adders and related gates implementing the partial-product reduction and final carry-propagate adder). Since all other blocks are either simpler control logic or already pipelined, the multiplier's internal combinational delay dominates, so the overall critical path is in the multiplier.

4.4 Throughput Analysis

Table 4.2: Throughput for 2D Convolution

	Design 1 (INW=18, R=9, C=8, MAXK=5)		Design 2 (INW=24, R=16, C=17, MAXK=9)	
TVALID and TREADY prob.	Cycle Count (cycles)	Throughput (ops/sec)	Cycle Count (cycles)	Throughput (ops/sec)
0.10	13,349,284	468,190	73,410,871	75,678
0.50	6,645,210	940,527	49,302,986	112,682
1.00	5,853,973	1,067,651	46,402,616	119,725

Throughput = 10000 ops / (total cycles for 10000 convolutions) * (1 / period)

4.5 Average Convolution Delay

Table 4.3: Delay for 2D Convolution

	Design 1		Design 2	
TVALID and TREADY prob.	Cycle Count (cyc/conv)	Average Delay (μ s)	Cycle Count (cyc/conv)	Average Delay (μ s)
0.10	1335	2.14	7341	13.21
0.50	665	1.06	4930	8.87
1.00	585	0.94	4640	8.35

Delay = (total cycles for 10000 convolutions) / 10000 * period

4.6 Average Energy Consumption

Table 4.4: Energy for 2D Convolution

TVALID and TREADY prob.	Design 1 Energy (nJ)	Design 2 Energy (nJ)
0.10	19.71	493.67
0.50	9.81	331.55
1.00	8.65	312.05

Energy = Power * Delay

4.7 Area-Delay Product

Table 4.5: Area-Delay Product for 2D Convolution

TVALID and TREADY prob.	Design 1 ADP (μ m ² ·ms)	Design 2 ADP (μ m ² ·ms)
0.10	40.36	988.24
0.50	20.09	663.71
1.00	17.70	624.66

Area-Delay Product = Area * Delay

4.8 Contribution

Jing completed the report section and verified the code correctness. Xiaoying drafted the code and handled data calculation.

Part 5: Throughput Optimization

5.1 The Optimized Conv Documentation

5.1.1 Module Overview

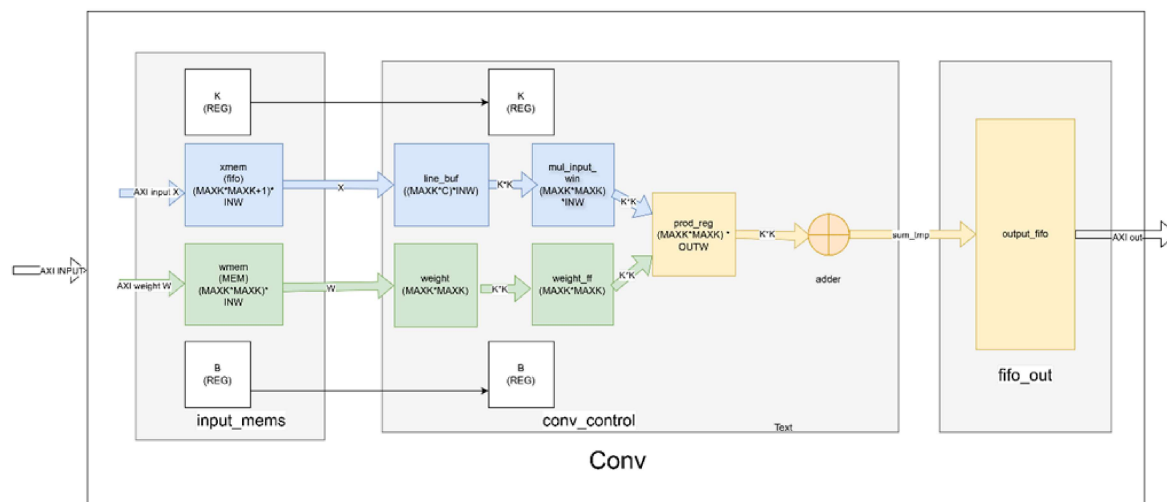


Figure 5.1. Optimized Conv Top-Level View

In my optimized design, the accelerator is still split into three blocks—**input_mems**, **conv_control**, and **out_fifo**—but they now form a three-stage streaming pipeline. The AXI interface first receives W , B , and X ; the weights are stored in a $\text{MAXK} \times \text{MAXK}$ weight memory, while the input image X is written into a small FIFO of depth $\text{MAXK} \times \text{MAXK} + 1$, once one image is loaded, the InputMems block can immediately start accepting the next one without waiting for the next stage to finish computing. When the fifo that stores input X is not empty, the **conv_control** stage begins reading pixels from the FIFO and weights from memory, filling a line buffer ($\text{MAXK} \times C$) until it can form a $K \times K$ window, then repeatedly slides this window, computes all $K \times K$ products in one cycle, and sums them in the next cycle to produce each output pixel. These outputs are written into an output FIFO in the **out_fifo** stage, which streams them out while the earlier stages continue working. As a result, loading, computing, and writing outputs can all proceed in parallel, and the overall throughput is determined by the slowest of the three pipeline stages instead of the sum of their latencies.

5.1.2 Techniques used to promote performance

a. Separating input_mems and conv_control

In the optimized design, InputMems and conv_control are cleanly separated:

InputMems only handles receiving W/B/X from AXI and storing them (W in a MAXK×MAXK RAM, X in an input FIFO), while ConvolutionControl independently reads W and X from these local buffers to perform the convolution. As a result, the AXI input stage no longer has to wait for the compute stage, and the two can run in parallel.

Why this improves performance:

- Overlap communication and computation
 - Previously: AXI receive → store all data → compute → only then receive next input.
 - Now: as soon as the FIFO has enough data, the compute engine starts; while it is working, the AXI interface can already stream in later pixels.
This removes long idle periods on the AXI port and increases effective throughput.
- Hide weight/bias latency
 - By making the FIFO depth MAXK*MAXK + 1, the pipeline can absorb the delay of fetching all K^2 weights and the bias.
 - The compute core sees a continuous stream of input pixels, so it is much easier to keep it busy every cycle. This hides the waiting time at the output interface of input_mems.
- Simpler control / shorter critical control paths
 - The new FSM no longer needs to coordinate a “load-all-then-compute” phase; instead, the interface follows a streaming, FIFO-based handshake.
 - This reduces control complexity and helps timing closure on the control path.

b. Line-buffered reuse of input rows

The detailed work flow is as follows (take a 5×5 inputX and 3×3 W as an example):

Step 1 – First complete window appears

The input pixels stream into the 3×5 line buffer row by row. At first the buffer is only partially filled and we cannot form a full 3×3 window. Once the third row has arrived and we have received pixels up to the third column, the line buffer contains a complete 3×3 block. At this moment the core can start the first convolution: it reads these 3×3 values from the line buffer together with the 3×3 weights and computes the first output for this row.

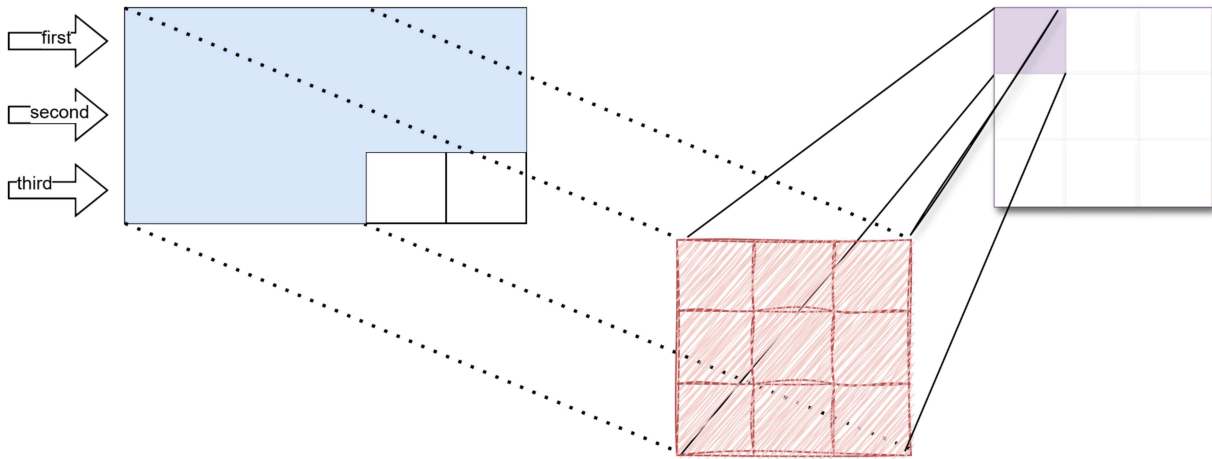


Figure 5.2. First Output Y[0,0]

Step 2 – Slide the window one column to the right

After the first output has been computed, the convolution window moves one column to the right inside the line buffer. Almost all of the data is reused: the top, middle, and bottom rows of the window stay the same, only the leftmost column is dropped and one new column on the right is read from the buffer. The core multiplies this new 3×3 block with the weights to produce the next output element in the same output row.

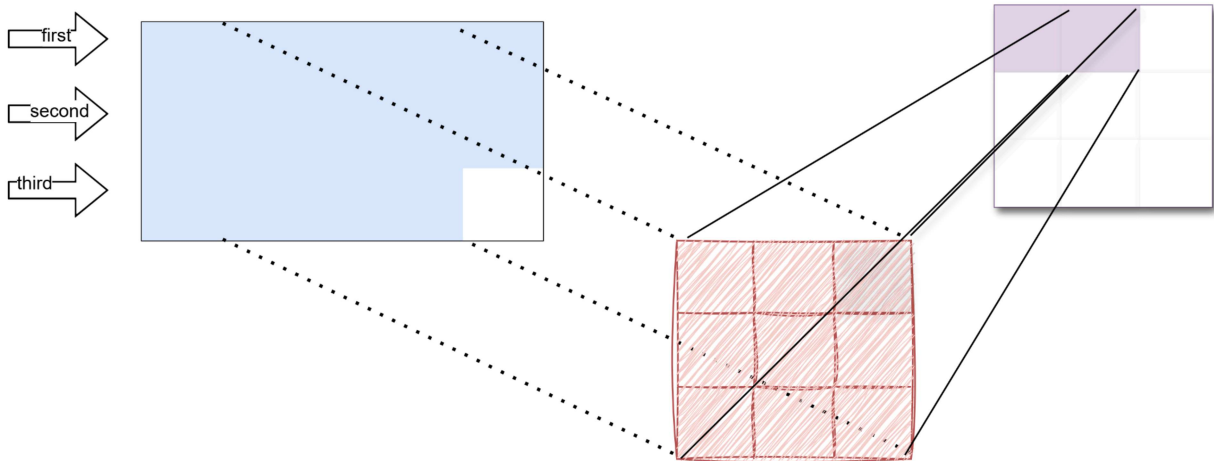


Figure 5.3: Second Output $Y[0,1]$

Step 3 – Slide to the last valid column in this row

The window continues to slide horizontally across the line buffer until it reaches the last valid 3×3 position (in this example, the rightmost place where three consecutive columns still fit inside the 3×5 buffer). After this step, all output pixels for the current output row have been generated using the current contents of the line buffer, so horizontally this line buffer configuration is “finished”.

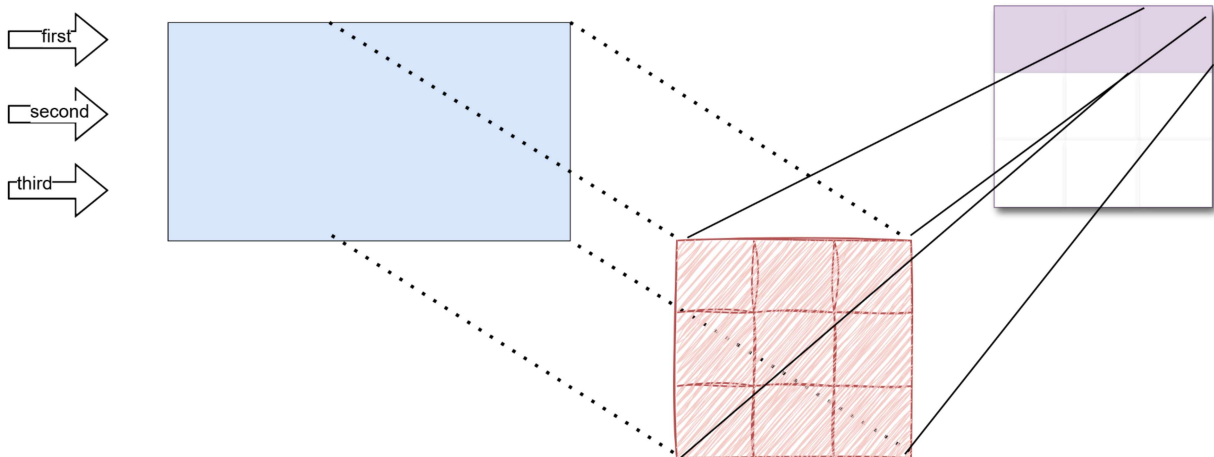


Figure 5.4. Third Output $Y[0,2]$

Step 4 – Overwrite the first buffer row with a new input row

To move the convolution down by one row, the next input row is streamed in and overwrites the oldest row in the line buffer (for example, the first/top row). The window will again start at the leftmost 3×3 position using these updated rows.

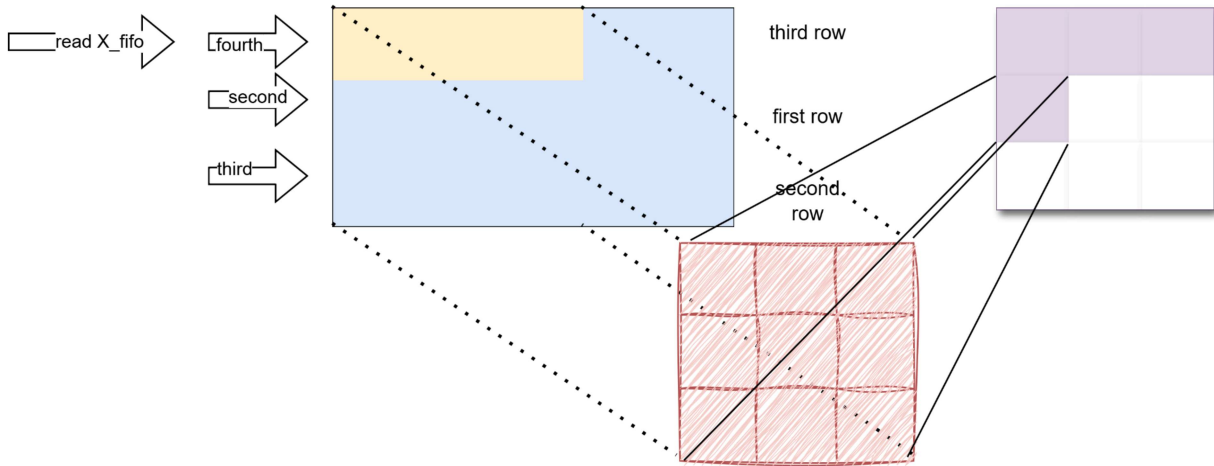


Figure 5.5. Overwrite the First Buffer Row

Step 5 – Overwrite the second buffer row with the next new row

As the input continues streaming, the following input row overwrites the next oldest buffer row (for example, the second row). After this update, the line buffer again holds the latest three rows needed for the next set of 3×3 windows. From this point the same pattern repeats: slide the window across the row (as in Steps 2–3), then overwrite the oldest buffer row when moving down to the next output row (as in Steps 4–5).

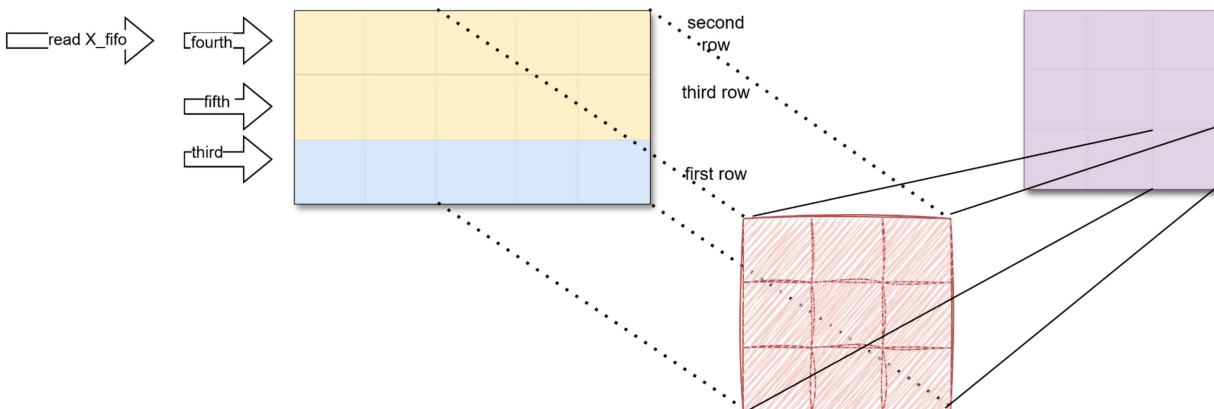


Figure 5.6. Overwrite the Second Buffer Row

Why this improves performance:

- **Data reuse**

Each input pixel is read once into the line buffer and reused for many overlapping $K \times K$ windows, instead of being reread from memory up to K^2 times.

- **Regular sliding-window dataflow**

The window just shifts inside the buffer (one new column per step), so we avoid issuing $K \times K$ separate reads and complex address calculations for every output.

- **High MAC utilization**

Once the buffer is filled, the core can form a valid $K \times K$ window almost every cycle, keeping the $K \times K$ multipliers busy with minimal stalls.

- Latency hiding and local accesses

After filling, convolution works on fast local storage (the line buffer), so external/AXI latency is mostly hidden and does not limit per-output throughput.

5.2 Cycle Count

5.2.1 input_mems

For the input_mems module, we can conceptually divide the input phase into three parts in the New K case:

- Number of weight words:

$$N_W = K^2$$

- Number of bias words:

$$N_{\text{bias}} = 1$$

- Number of input pixels:

$$N_X = R \cdot C$$

Theoretically, new K needs an extra $K^2 + 1$ cycles compared with non-NewK, purely to load the new weights and bias.

Streaming view with FIFO (what our design actually does)

In my optimized, streaming architecture, this overhead is largely hidden:

- The input pixels X are pushed into an input FIFO as a stream.
- At the same time, the weights and bias are written into the internal weight registers/RAM.
- Because the FIFO has enough depth, the weight/bias loading time is overlapped with the input streaming.

From this streaming perspective, the effective input time for both NewK and non-NewK is dominated by the time to receive the image:

$$T_{\text{in,eff}}^{\text{NewK}} \approx T_{\text{in,eff}}^{\text{nonNewK}} \approx N_X = R \cdot C$$

Due to the 1 cycle/row delay time (the following part), the final time in this stage is :

$$T_{\text{in,eff}}^{\text{NewK}} \approx T_{\text{in,eff}}^{\text{nonNewK}} \approx N_X = R \cdot (C + 1)$$

In other words, once we treat the system as a streaming pipeline, the New K and non-New K cases have essentially the same input-side latency, determined mainly by the size of the input feature map.

5.2.2 conv_control

For the compute part (line buffer, control, sliding window, multipliers, and accumulation), we can split the timing into:

1. Startup time until the first valid output appears.

$$T_{\text{fill}} = (K - 1) \cdot C + K$$

2. Total compute time until the last output is produced.

In my current control implementation, there is one extra subtlety:

- At the end of each output row, after launching the last window in that row, the controller waits one extra cycle for the final multiplication/accumulation to complete before it starts reading the first pixel of the next input row.
- This introduces a one-cycle bubble per output row.

The number of output rows is $R - K + 1$, so this behavior adds approximately

$$N_{\text{bubble}} = R - K + 1$$

extra cycles. Taking this into account, the total compute time from the first input pixel to the last output in the current design can be approximated as:

$$T_{\text{compute}} \approx R \cdot C + (R - K + 1).$$

5.2.3 overall time

For a single image, the total processing time is dominated by the slowest stage rather than by the sum of all three. so conv_control is usually the longest stage. Therefore, the effective processing time of the whole pipelined system is:

$$T_{\text{compute}} \approx R \cdot C + (R - K + 1).$$

5.3 Optimize 2D Convolution Synthesis Results

Table 5.1: Synthesis Results for Optimize 2D Convolution

Parameters	Period (ns)	Frequency (MHz)	Area (μm^2)	Power (mW)
INW=12, R=9, C=8, MAXK=5	2.0	500	47299.32	11.97
INW=18, R=9, C=8, MAXK=5	2.0	500	83620.56	17.86
INW=24, R=16, C=17, MAXK=9	4.7	212.77	412641.55	36.60

The critical path is identical across all configurations. In the optimized $K \times K$ design, the current critical path goes from a bit of the window register mul_input_win_reg, through the full combinational implementation of one multiplier (partial-product logic plus a deep adder tree), to the corresponding bit of prod_reg_reg. Although this looks like “just many multipliers in parallel”, the per-multiplier path is actually much heavier than in the original serial design: the new path includes additional selection/gating logic around the window registers, a larger internal adder tree, and higher output fanout. In contrast, the original mac_pipe had a very clean path from input registers straight into a standalone multiplier and then into pipe_reg, with almost no extra logic, so the single-cycle multiplier delay was much shorter.

We did not further optimize this critical path because the most obvious fix—introducing pipelined DesignWare multipliers—would increase the internal MAC latency and require additional pipeline stages in both the multiplier and the K^2 -term adder tree, while only providing a modest reduction in combinational delay. After evaluating this trade-off, I chose not to further pipeline the multiplier/accumulator, and instead focused on architectural optimizations (line buffer, streaming FIFO, window reuse) that gave a much clearer throughput improvement.

5.4 Throughput Analysis

Table 5.2: Throughput for Optimize 2D Convolution

	Design 1 (INW=18, R=9, C=8, MAXK=5)		Design 2 (INW=24, R=16, C=17, MAXK=9)	
TVALID and TREADY prob.	Cycle Count (cycles)	Throughput (ops/sec)	Cycle Count (cycles)	Throughput (ops/sec)
0.10	7,954,700	628,559	29,143,531	73,006
0.50	1,584,828	3,154,916	5,799,138	366,892
1.00	809,317	6,178,049	2,913,897	730,177

Throughput = 10000 ops / (total cycles for 10000 convolutions) * (1 / period)

5.5 Average Convolution Delay

Table 5.3: Delay for Optimize 2D Convolution

	Design 1		Design 2	
TVALID and TREADY prob.	Cycle Count (cyc/conv)	Average Delay (μ s)	Cycle Count (cyc/conv)	Average Delay (μ s)
0.10	795	1.59	2914	13.70
0.50	158	0.32	580	2.73
1.00	81	0.16	291	1.37

Delay = (total cycles for 10000 convolutions) / 10000 * period

5.6 Average Energy Consumption

Table 5.4: Energy for Optimize 2D Convolution

TVALID and TREADY prob.	Design 1 Energy (nJ)	Design 2 Energy (μ J)
0.10	28.41	501.33
0.50	5.66	99.76
1.00	2.89	50.12

Energy = Power * Delay

5.7 Area-Delay Product

Table 5.5: Area-Delay Product for Optimize 2D Convolution

TVALID and TREADY prob.	Design 1 ADP ($\mu\text{m}^2 \cdot \text{ms}$)	Design 2 ADP ($\mu\text{m}^2 \cdot \text{ms}$)
0.10	133.04	5652.14
0.50	26.50	1124.69
1.00	13.54	565.13

Area-Delay Product = Area * Delay

5.8 Comparison Between Optimized and Baseline Convolution Designs

Table 5.6: Percentage Change of Metrics (Optimize vs. Baseline)

Metrics	Design 1 (INW=18, R=9, C=8, MAXK=5)			Design 2 (INW=24, R=16, C=17, MAXK=9)		
	0.1	0.5	1.0	0.1	0.5	1.0
Throughout	34.25%	235.44%	478.66%	-3.53%	225.60%	509.88%
Delay	-25.51%	-70.19%	-82.72%	3.66%	-69.29%	-83.60%
Energy	44.13%	-42.31%	-66.56%	1.55%	-69.91%	-83.94%
ADP	229.65%	31.94%	-23.52%	471.94%	69.46%	-9.53%

Percentage change = (optimized - baseline) / baseline

Metrics Performance

Conv vs. Optimize Conv

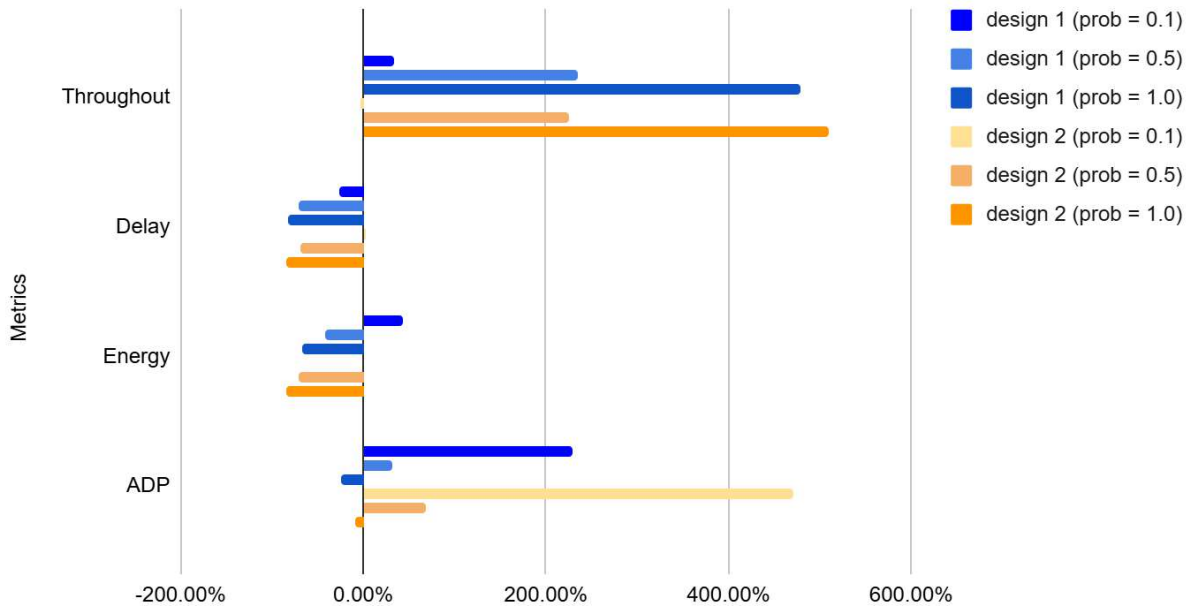


Figure 5.7. Percentage Change of Metrics (Optimized vs. Baseline)

Whether the speed-optimized design is “more efficient” than the Part-4 baseline actually depends on two things:

1. The switching probability used in the power model (Prob. = 0.1 / 0.5 / 1.0), and
2. The parameters INW, R, C, and MAXK of each design.

Role of Prob. (switching probability)

In the power analysis, “Prob.” controls the average activity of the data signals (e.g., probability that a bit is 1 / toggles).

Dynamic power and therefore energy scale roughly linearly with this probability:

- When Prob = 0.1, both the baseline and the optimized core switch relatively rarely. In this scenario, the serial Part-4 design already consumes little dynamic power, and the extra parallel hardware of the new design mainly shows up as extra capacitance and leakage. As a result, energy and ADP often get worse, even though throughput is somewhat higher (Design 1) or even slightly lower (Design 2).

- When Prob = 0.5 or 1.0, the datapath is very active.

Now the baseline serial MAC has to toggle for many more cycles to finish the same convolution, while the optimized design finishes in far fewer cycles due to the line buffer and parallel MACs. In this case the dynamic power increase from extra hardware is compensated (and more than compensated) by the large reduction in cycle count, so energy per image goes down by 40–80% and throughput goes up by 2–5×. That is why at high Prob the new design becomes clearly more efficient.

Thus,

- low activity → area overhead dominates (less efficient);
- high activity → cycle reduction dominates (more efficient).

Role of the design parameters (Design 1 vs Design 2)

The same effect also depends on the convolution size:

- Design 1: INW = 18, R = 9, C = 8, MAXK = 5
(smaller image and kernel)
- Design 2: INW = 24, R = 16, C = 17, MAXK = 9
(larger image and kernel, many more MAC operations per output)

For larger R, C, and MAXK (Design 2), each convolution requires many more K^2 MACs, so:

- The compute time of the baseline grows quickly, while
- The optimized design can reuse data through the line buffer and compute multiple products in parallel.

In summary, the speed-optimized design is most efficient for large, dense convolutions (larger R, C, MAXK and higher Prob.), where the reduced cycle count dominates, but it can be less efficient for small or very sparse workloads, where the extra hardware mainly adds area and energy without a proportional speedup.

5.9 Contribution

Jing completed the report section and handled the code. Xiaoying performed data analysis.